Author

Kshitij Sinha

24f3004673

Joined the online programs since 2024 has been a great journey so far, I genuinely enjoyed learning new concepts and this is not just for formality.

Description

To be honest doing the project gave a better understanding of the concepts of the modern application development one course. In this we had to make a website for hospital management through which hospital can seamlessly manage all its task.

Technologies Used

Flask**:** was used to develop the link routes and URL

SQL Alchemy: used to play with the database and integrating it seamlessly.

HTML: Used to make the structure of the front end and for some styling to add some aesthetic appeal

SQLite: The main backbone of the entire project in absence of which there will be no purpose to the project it will be just a normal website with text without any meaning

DB schema design

Admin Table

admin_ID: primary key, auto-incremented, not null; name: not null; password: not null, unique

Doctor Table

doctor_ID: primary key, auto-incremented, not null; name: not null; specialisation: not null; availability: nullable; password: not null; contact: not null

Patient Table

patient_ID: primary key, auto-incremented, not null; name: not null; contact_info: not null; login: nullable; password: not null; age: nullable

Department Table

department_ID: primary key, auto-incremented, not null; name: not null; description: nullable

Appointment Table

appointment_ID: primary key, auto-incremented, not null; patient_ID: not null; doctor_ID: not null; date: not null; time: not null; status: not null

Treatment Table

appointment_ID: primary key, auto-incremented, not null; diagnosis: nullable; prescription: nullable; notes: nullable; symptoms: nullable

Blacklist Table

id: primary key, auto-incremented, not null; name: not null

API Design

In this project, we created different API routes using GET and POST methods to manage data between pages and the database. These routes help in adding, viewing, and updating information like doctors, patients, and appointments.

Architecture and Features

Architecture

- main.py handles all backend routes.
- models.py defines all database structures.
- templates folder contains HTML pages with inline styling.

Features

- **Admin**: Log in, manage doctors, patients, appointments, and treatment history.
- **Doctor**: Log in, view assigned appointments, update treatment details.
- **Patient**: Log in, view appointments and previous prescriptions.
- Supports appointment booking and viewing history with easy navigation.

Video

https://drive.google.com/file/d/1WkOOl8Xi_gfySkEzKkOG4QNZVofTOIqt/view?usp=sharing